SESSION 2 REFERENCE GUIDE: CONDITIONAL LOGIC, LOOPS & DATA VALIDATION

This guide provides a reference for comparison operators, conditional logic branching, functions, `for` loops, and error handling syntax introduced in Session 2.

1. Comparison & Logical Operators

Comparison operators evaluate expressions and return Boolean values (`True` or `False`).

Operator	Meaning	Example	Result
`==`	Equal to	`18.5 == 18.5`	`True`
`!=`	Not Equal to	`"Express" != "Standard"`	`True`
`>`	Greater than	`26.5 > 24.0`	`True`
`<`	Less than	`8.5 < 12.0`	`True`
`>=`	Greater than or equal to	`12.0 >= 12.0`	`False`
`<=`	Less than or equal to	`14.0 <= 12.0`	`False`

Combining Conditions with Logical Operators (`and`, `or`, `not`)

```Python
# Both conditions must be True for the statement to be True
if order_type == "Express" and processing_hours > 12.0:
    print("Express SLA Breach")

# At least one condition must be True
if processing_hours > 24.0 or order_status == "CANCELLED":
    print("Requires Escalation")
```

2. Conditional Branching (`if`, `elif`, `else`)

Python executes code blocks based on top-down evaluation of conditions.

```Python
processing_hours = 18.5

if processing_hours > 24.0:
    category = "Severe Delay"
elif processing_hours > 12.0:
    category = "Moderate Delay"
else:
    category = "On Schedule"

# Result: category = "Moderate Delay"
```

3. Creating Functions (def)

Functions encapsulate logic into reusable blocks. They take inputs (parameters) and return an output (`return`).

```Python
def calculate_sla_gap(actual_hours, limit_hours):
    """Calculates hours over or under target SLA limit."""
    gap = actual_hours - limit_hours
    return gap

# Function Call Example
variance = calculate_sla_gap(28.0, 24.0) # Returns 4.0
```

4. Iteration with for Loops

A `for` loop iterates through each item in a list or collection.

```Python
processing_times = [12.5, 8.0, 26.0, 14.5]

for hours in processing_times:
    if hours > 24.0:
        print(f"Hours: {hours} -> BREACH")
    else:
        print(f"Hours: {hours} -> OK")
```

5. Defensive Programming: Error Handling (try / except)

Use `try` / `except` blocks to handle corrupted or unexpected data values without stopping script execution.

```Python
raw_value = "CORRUPTED_ENTRY"

try:
    clean_hours = float(raw_value)
    print("Conversion successful:", clean_hours)
except ValueError:
    print("Data Error: Unable to convert raw value to numeric float.")
    clean_hours = None
```

6. The Analyst Validation Cycle

When evaluating business datasets, execute this defensive sequence:

```
[Raw Input] --> [Try Float Parsing] --> [Validate Business Range] -->
[Apply Conditional Logic] --> [Update Aggregates]
```